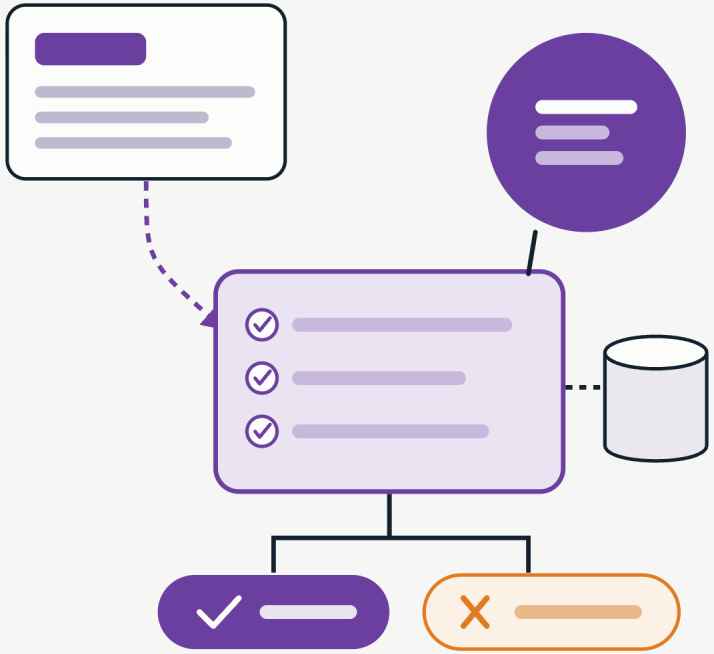
ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО
ЕЛЕМЕНТИ РАЗВОЈА СОФТВЕРА

Пословна логика и слој случајева употребе

Распоређивање пословних правила и организација сценарија случаја употребе.



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРЕГЛЕД

Садржај презентације

01

Модел и правила

Модел, валидатор и доменски сервис, factory метод и ограничења валидног стања.

04–07

02

Исходи и Result

Очекивани неуспех, правило преклапања, Result тип и стабилни кодови исхода.

08–13

03

Сервис случаја употребе

Оркестрација, dependency injection, контрола времена и положај правила.

14–21

04

Контролна тачка КТ1

Вертикални пролаз, захтеви КТ1, самостални задатак и припрема.

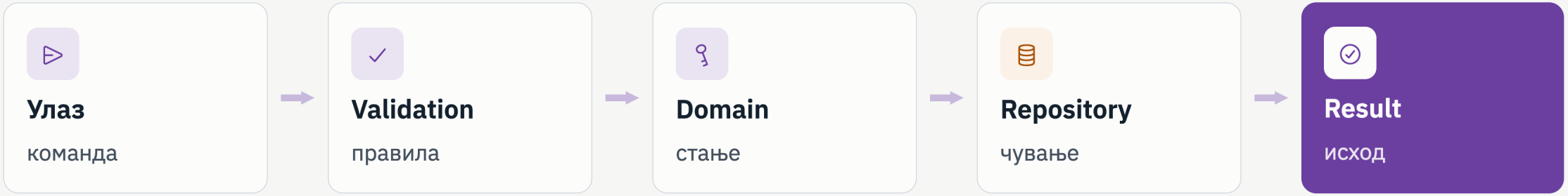
22–27

Увод · ток случаја употребе од улаза до исхода (03)

УВОД

Случај употребе и пословни исход

Случај употребе оркестрира валидаторе, доменске сервисе и портове без зависности од корисничког интерфејса и базе.



НАПОМЕНА

Случај употребе има јасан улаз, експлицитан резултат и контролисане зависности.

ПРВИ ДЕО

01

Модел и правила

Ко носи стање, ко проверава вредности, а ко спроводи правила сценарија.

Модел | Валидатор | Доменски сервис | Factory метод | Ограничења

01 · МОДЕЛ И ПРАВИЛА

Модел и пословна правила

Модел представља стање, док валидатори и доменски сервиси проверавају ограничења и спроводе пословна правила.

Случај употребе координише валидаторе, доменске сервисе и портове

МОДЕЛ

Reservation

садржи податке о периоду и стању

ВАЛИДАТОР

ReservationValidator

проверава дозвољене вредности

ДОМЕНСКИ СЕРВИС

правила сценарија

спроводи правила која обухватају више објеката

Factory метод и валидно стање

Factory метод користи валидацију приликом креирања модела и враћа експлицитан Result.

C# · Domain

```
1   public sealed class ReservationMaker
2   {
3       public Result<Reservation> Create(DateTime from, DateTime to)
4       {
5           if (from >= to)
6               return Result<Reservation>.Fail("InvalidPeriod");
7
8           return Result<Reservation>.Ok(
9               new Reservation(from, to));
10      }
11  }
```

НАПОМЕНЕ

Конструктор модела може бити internal, па модел настаје само кроз factory.

Factory спречава очигледно невалидно стање.

Ентитет чува сопствене инваријанте, а правила сценарија остају у сервисима.

Ограничења валидног модела

Валидатор проверава ограничења вредности пре покретања пословног сценарија.

01

Период

почетак мора бити пре краја

`InvalidPeriod`

02

Стање

отказана резервација се не отказује поново

`AlreadyCancelled`

03

Време

прошла резервација се не мења као будућа

код дефинише тим за свој домен

ДРУГИ ДЕО

02

Исходи и Result

Како систем исправно каже „не” и како клијент поуздано разуме зашто.

- Очекивани неуспех
- Правило преклапања
- Result тип
- Избор модела исхода
- Кодови исхода

Очекивани пословни неуспех

Заузет термин представља исправно одбијен захтев са стабилним кодом исхода.

> **Систем ради исправно када стабилно врати OverlappingReservation и не промени стање које не сме да се промени.**

ЗАХТЕВ

термин се преклапа са постојећом резервацијом

→

ИСХОД

`Fail("OverlappingReservation")`

→

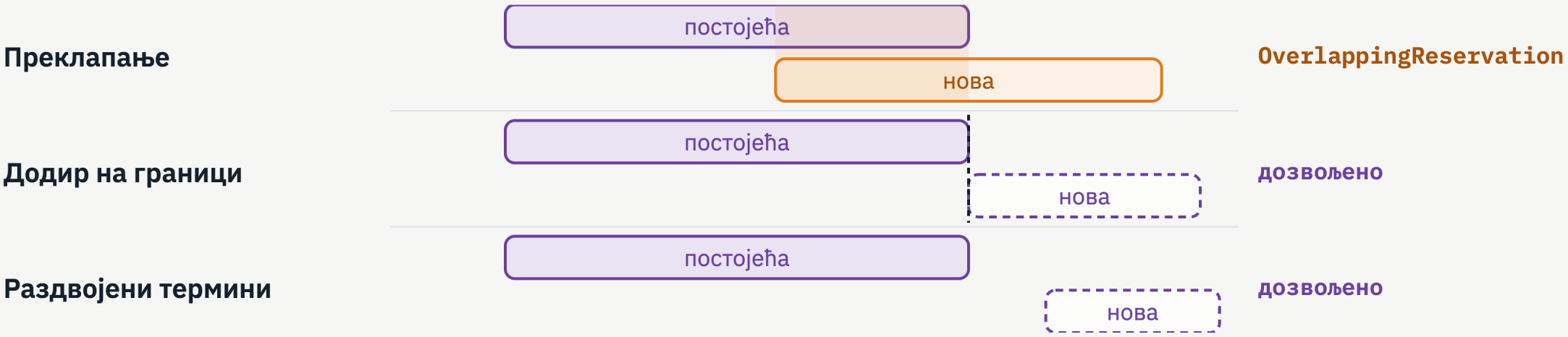
СТАЊЕ

непромењено, ништа није сачувано

Правило преклапања термина

Термин је полуотворен интервал [from, to): крај једне резервације може бити почетак друге.

ПРЕКЛАПАЊЕ $a.\text{from} < b.\text{to} \land b.\text{from} < a.\text{to}$



Result као модел исхода

Result експлицитно представља успех или очекивани пословни неуспех.

C# · Application

```
public sealed record Result(bool Success, string? Error)
{
    public static Result Ok() => new(true, null);
    public static Result Fail(string error) => new(false, error);
}

public sealed record Result<T>(bool Success, T? Value, string? Error)
{
    public static Result<T> Ok(T value) => new(true, value, null);
    public static Result<T> Fail(string error) => new(false, default, error);
}
```

Код неуспеха је стабилан.

Клијент не мора да тумачи изузетак.

Тест директно проверава очекиван исход.

Избор модела исхода

Модел исхода зависи од врсте грешке и места на коме она настаје.

Ситуација	Препоручени модел
Неважећи пословни захтев	Result и стабилан код неуспеха
Ентитет није пронађен	NotFound исход или Result код
База недоступна	инфраструктурна грешка
Нарушена доменска претпоставка	експлицитан неуспех у пословном слоју

Плаво: пословни исход који клијент обрађује.

Наранџасто: техничка грешка изван пословног тока.

Кодови исхода као део уговора

Стабилни кодови омогућавају клијенту да поуздано обради очекивани неуспех.

Код	Значење
InvalidPeriod	почетак није пре краја
ReservationMustBeInFuture	термин није у будућности
OverlappingReservation	период се преклапа
AlreadyCancelled	операција је већ извршена

НАПОМЕНА

Код се не мења када се промени текст поруке, па клијент и тестови остају стабилни.

ТРЕЋИ ДЕО

03

Сервис случаја употребе

Оркестрација сценарија, зависности које се прослеђују споља и место сваког правила.

ReservationService | Dependency Injection | IClock | Контролер и use-case | Правила по слојевима

Сервис случаја употребе

Сервис координише сценарио преко доменских сервиса и уговора.

C# · Application

```
1   public sealed class ReservationService
2   {
3       private readonly IReservationRepository _repository;
4       private readonly IClock _clock;
5       public Result Reserve(ReservationRequest request)
6       {
7           if (request.From <= _clock.UtcNow)
8               return Result.Fail("ReservationMustBeInFuture");
9           if (_repository.HasOverlap(request.EquipmentId,
10              request.From, request.To))
11              return Result.Fail("OverlappingReservation");
12          // креирање преко factory метода и чување у repository
13          return Result.Ok();
14      }
15  }
```

НАПОМЕНЕ

- Use-case проверава ток операције.
- Repository је уговор, а не конкретна база.
- Време је зависност која се контролише.

03 · СЕРВИС СЛУЧАЈА УПОТРЕБЕ

Dependency Injection

Промењљиве зависности прослеђују се споља преко јасних уговора.

01

IClock

контролише време

02

Repository

складиште као уговор

03

Service

оркестрација use-case-а

04

Composition root

бира конкретне имплементације

НАПОМЕНА

Сервис добија зависности кроз конструктор и не креира их сам.

03 · СЕРВИС СЛУЧАЈА УПОТРЕБЕ

Време као контролисана зависност

Правило „термин мора бити у будућности” постаје поновљиво тек када време долази преко уговора.

C# · Application и Tests

```
1   public interface IClock
2   {
3       DateTime UtcNow { get; }
4   }
5   // у раду апликације: стварно време
6   public sealed class SystemClock : IClock
7   {
8       public DateTime UtcNow => DateTime.UtcNow;
9   }
10  // у тесту: фиксно и поновљиво време
11  public sealed class FixedClock(DateTime now) : IClock
12  {
13      public DateTime UtcNow => now;
14  }
```

НАПОМЕНЕ

DateTime.Now у сервису чини тест зависним од тренутка покретања.

Апликација користи SystemClock, а тест FixedClock.

Од .NET 8 постоји и уграђени TimeProvider са истом улогом.

03 · СЕРВИС СЛУЧАЈА УПОТРЕБЕ

Положај пословних правила

Контролер преводи улаз, позива случај употребе и мапира резултат.

Controller

- прима захтев
- валидација формата
- позива application use-case
- мапира Result у HTTP или UI одговор

Use-case

- проверава пословни ток
- позива доменске операције
- користи портове
- враћа експлицитан исход

03 · СЕРВИС СЛУЧАЈА УПОТРЕБЕ

Граница случаја употребе

Случај употребе представља операцију са јасним улазом, пословним исходом и зависностима.

> **Добар use-case може да се објасни од захтева до резултата.**

НАМЕРА
шта корисник покушава

ПРАВИЛА
која правила важе

СТАЊЕ
које стање се мења

ИСХОД
који исход систем враћа

НАПОМЕНА
Случај употребе је јединица пројектовања и одбране функционалности.

03 · СЕРВИС СЛУЧАЈА УПОТРЕБЕ

Распоређивање правила по слојевима

Положај правила зависи од података које користи и стања које штити.

01	**Entity**	инваријанте сопственог стања
02	**Domain service**	правило више доменских објеката
03	**Application service**	оркестрација use-case-а
04	**Infrastructure**	технички адаптер без пословне одлуке

Што је правило ближе језгру, то ређе треба да се мења и мање зависи од технологије.

03 · СЕРВИС СЛУЧАЈА УПОТРЕБЕ

Границе случаја употребе

Јасан улаз, пословно правило и експлицитан резултат омогућавају проверу сценарија.

01

Input

команда или захтев

`ReservationRequest`

02

Policy

пословно понашање

`будућност, преклапање`

03

Output

експлицитан исход

`Result`

ЧЕТВРТИ ДЕО

04

Контролна тачка КТ1

Шта функционално језгро мора да покаже и како се припремамо за тестирање.

Вертикални пролаз | Захтеви КТ1 | Самостални задатак | Резиме | Припрема

04 · КОНТРОЛНА ТАЧКА КТ1

Контролна тачка КТ1

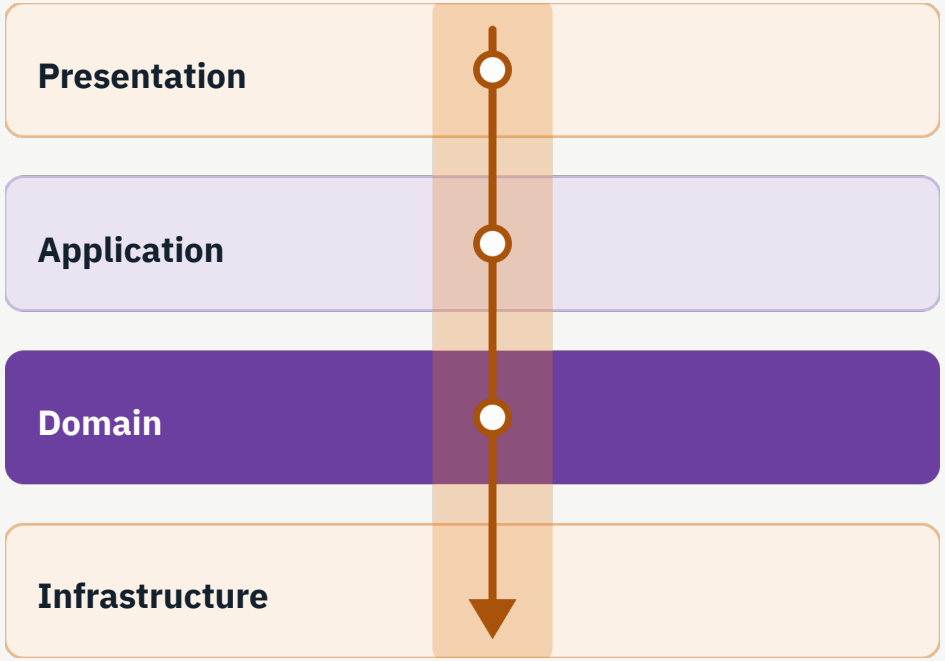
Функционално језгро мора бити разумљиво и образложиво.

1

ВЕРТИКАЛНИ ПРОЛАЗ

Најмање један пролаз кроз систем, од захтева до резултата, са јасним границама одговорности.

КТ1 је у термину Вежбе 4 и носи највише 10 поена.



04 · КОНТРОЛНА ТАЧКА КТ1

Захтеви контролне тачке КТ1

Провера обухвата архитектонске границе, вертикални пролаз и стабилне исходе.

01	Документовати одговорност сваког слоја и смер зависности.
02	Имплементирати најмање један вертикални пролаз кроз систем.
03	За очекиване неуспехе дефинисати стабилне кодове или типове резултата.
04	Domain и Application тестирати без базе и корисничког интерфејса.
05	Термин: Вежба 4 · највише 10 поена.

04 · КОНТРОЛНА ТАЧКА КТ1

Самостални задатак: bool/null у Result

Метода треба да врати стабилан код и податке потребне за обраду исхода.

C# · пре и после

```
// пре
bool Reserve(Request request);

// после
Result Reserve(Request request);

// тест проверава
Assert.That(result.Success, Is.False);
Assert.That(result.Error,
    Is.EqualTo("OverlappingReservation"));
```

ЗАДАТАК

- Рефакторисати једну методу са bool/null исходом.
- Додати два мала теста или демонстрациона сценарија.
- Образложити нови стабилан код неуспеха.

04 · РЕЗИМЕ

Пословна логика припада доменским сервисима и случајевима употребе, уз одвојену валидацију модела.

Use-case није техничка основа, већ контролисан ток који повезује захтев, доменска правила, зависности и стабилан резултат.

захтев → правила → зависности → Result

Овим се поставља основа за аутоматизовано тестирање.

04 · КОНТРОЛНА ТАЧКА КТ1

Припрема за наредну вежбу

Припремити сценарије који се могу проверити NUnit тестовима.

01

Изабрати два use-case-а у пројекту.

02

За сваки дефинисати улаз, пословне исходе и стабилне кодове грешке.

03

Проверити да ли доменски објекти штите своје инваријанте.

04

Изместити пословна правила из UI/API слоја.

05

Припремити сценарије за NUnit тестове.

Гранични случај из правила преклапања (слајд 10) је добар први кандидат за тест.